GLOSSARY

Alphabetical. Each entry points to the lesson where the term is taught in depth. "Impl." marks implementation details that can change between React releases.

Term	Definition	Lesson
Abandoned render	A render that ran but was never committed (superseded or interrupted).	02/15
Action (React 19)	A function run inside a transition, possibly async; tracked for pending state, errors, and form reset.	03/22
Adapter / port	A port is the interface the app needs; an adapter implements it for a concrete technology.	05/15
Anti-corruption layer	Mapping code that keeps external wire formats from leaking into the domain model.	04/14
asChild / Slot	Pattern merging a component's props and ref onto its single child instead of rendering a wrapper.	05/20
Automatic batching	Grouping state updates from any source (events, timers, promises) into one render.	02/22
Bailout	Skipping a component or subtree because nothing relevant changed.	02/29
Blast radius	The set of components that re-render when a piece of state changes.	07/07
Browser engine	Component implementing parsing, DOM, style, layout, paint, compositing (Blink, Gecko, WebKit).	00/01
Closure	A function together with the lexical environment it was created in.	00/06
Commit phase	Synchronous, uninterruptible phase where React mutates the host (DOM) and runs layout effects.	02/11
Compositor	Thread that assembles layers into frames; can scroll/animate some properties without the main thread.	00/03
Composition root	The single place where concrete implementations are chosen and wired.	05/16
Compound components	Components sharing implicit state (usually via context) to form one API.	05/05
Controlled / uncontrolled	Value owned by React state (`value` + `onChange`) vs by the DOM (`defaultValue`).	01/11
Core Web Vitals	Google's user-centric metrics (LCP, INP, CLS). Thresholds change; verify on web.dev.	07/21
DAL / DTO	Data Access Layer (auth + shaping in one module); Data Transfer Object (explicit projection safe to send to the client).	09/12
Derived state	A value computed during render rather than stored.	03/17
Discriminated union	A union tagged by a literal property, enabling exhaustive narrowing.	00/18
Effect Event	`useEffectEvent` function callable only from Effects that always reads latest values (⚠ verify status).	03/22

Term	Definition	Lesson
Element (React)	Plain immutable object `{type, props, key}` describing UI.	01/04
Event delegation (React)	One set of listeners on the root container dispatches to React handlers.	01/10
Expand/contract	Change a contract by supporting old and new together, migrating callers, then removing the old.	10/12
Fiber	React's internal unit of work and data structure per tree position (Impl.).	02/09
Fitness function	Automated check that an architectural property still holds.	04/25
Forced synchronous layout	Reading layout-dependent values while layout is dirty, forcing immediate layout.	00/03
Headless component	Behavior and accessibility without markup or styles.	05/07
Hydration	Attaching React to server-rendered HTML so it becomes interactive.	02/25
Idempotency key	Client-generated token letting a server deduplicate retried writes.	06/15
Input delay	Time from an input event until its handler starts (main thread busy).	00/01
Jank	Visible stutter from missed frames.	00/01
Key	Identity claim for an element among siblings; changing it remounts.	02/07
Lane	A bit naming a channel of pending work with a priority (Impl.).	02/14
Long task	Main-thread task of 50 ms or more.	07/20
Main thread	Per-document thread running JS, style, layout, paint recording, input dispatch.	00/01
Memoization	Caching a result keyed by inputs. In React a performance hint, not a semantic guarantee.	03/11
Optimistic overlay	Temporary view of state (`useOptimistic`) reverting when the Action settles.	06/05
Prop getter	Function returning the merged props an element needs for a behavior.	05/06
Purity	Output depends only on inputs; no observable side effects. React expects render to be pure.	01/12
Ratchet	Enforcement baseline that may only improve.	04/25
Rebasing	Replaying skipped lower-priority updates in order on a base state.	02/14
Reconciliation	Matching new elements against existing fibers to decide reuse, move, create, delete.	02/06
Referential equality	Two references are equal only if they point to the same object (`Object.is`).	00/07
Render phase	React calling components to compute the next tree; pure, interruptible, repeatable.	02/11
RSC payload	Streamed serialized tree produced by Server Components (wire format is Impl.).	02/27
Scheduling	Deciding when and in what order pending updates are processed.	02/13
Selective hydration	Hydrating Suspense boundaries independently, prioritized by interaction.	02/25
Server state	Client snapshot of data owned remotely; needs caching, invalidation, refetch.	06/07

Term	Definition	Lesson
Snapshot (state)	Fixed props, state, and handlers of one render.	01/14
Stale closure	A callback from an older render still reading that render's values.	03/05
State colocation	Keeping state in the lowest component that needs it.	03/18
Strangler fig	Incrementally routing traffic from a legacy system to its replacement.	10/13
Structural sharing	Updates that reuse unchanged subtrees' references.	00/08
Suspense	Boundary that shows a fallback while descendants wait (e.g. on `use` or lazy code).	02/24
Tearing	One render seeing different values of the same external store.	03/20
Time slicing	Splitting render work into short chunks that yield to the browser.	02/15
Transition	Update marked non-urgent so it can be interrupted by urgent ones.	02/23
Version skew	Old client running against a newer deployment.	10/02
Virtual DOM	A teaching term for React's in-memory description of UI; not a distinct technology.	02/04
Web API / host API	Functionality supplied to JS by the host (DOM, `fetch`, timers).	00/01
Windowing	Rendering only visible rows plus overscan.	07/09